

SOFIAA OS

Sistema Operativo de Inteligencia Artificial

ALEJANDRÍA

Knowledge Graph de Autoconocimiento de Ingeniería

Reporte Técnico Integral — Nivel Doctoral

Versión 1.0 · Julio 2026 · Confidencial

Arquitectura: Next.js 14 · TypeScript · Firebase · Gemini · Vercel

Tabla de Contenidos

1. Introducción y Contexto Arquitectónico
2. Filosofía de Diseño — El Problema del Autoconocimiento
3. Arquitectura del Corpus
4. Sistema de Tipos — Schema Completo
5. Pipeline de Ingestión (AJ-1)
6. Motor de Búsqueda Semántica (AJ-2)
7. Inyección de Contexto en el Chat (AJ-3)
8. Extension Tools (AJ-4)
9. Interfaz de Usuario (AJ-5)
10. Esquema Firestore
11. Seguridad y Control de Acceso
12. Capa de API REST
13. Algoritmo de Scoring Híbrido
14. Historia de Sprints
15. Roadmap
16. Conclusiones

Resumen Ejecutivo

ALEJANDRÍA es el sistema de autoconocimiento de ingeniería del ecosistema SOFIAA OS, un sistema operativo de inteligencia artificial modular construido sobre Next.js 14, TypeScript, Firebase y modelos de lenguaje de gran escala. ALEJANDRÍA convierte la documentación histórica del propio sistema —decisiones de arquitectura, reportes de sprint, especificaciones de módulos, brainstormings y experimentos— en un Knowledge Graph semántico consultable en tiempo real.

La premisa central de ALEJANDRÍA es radical: un sistema de IA suficientemente maduro debe ser capaz de razonar sobre sí mismo con la misma precisión con que razona sobre cualquier otro dominio. Cuando un usuario pregunta '¿cómo funciona NEXO?', '¿por qué se eligió Groq en lugar de OpenAI?', o '¿qué se construyó en el Sprint A-9?', ALEJANDRÍA proporciona la respuesta desde su corpus de fuente de verdad —no desde la memoria paramétrica del modelo de lenguaje, que puede alucinar o quedar desactualizada.

El diseño arquitectónico de ALEJANDRÍA parte de tres principios fundamentales: (1) Corpus como Fuente de Verdad —59 documentos JSON estructurados, generados determinísticamente desde la documentación original, constituyen la base inamovible del conocimiento; (2) Búsqueda Híbrida —el motor combina similitud coseno sobre embeddings de 768 dimensiones (Gemini gemini-embedding-001) con scoring por keywords y reinforcement learning por uso; y (3) Zero-Latency Context Injection —el contexto de ALEJANDRÍA se inyecta automáticamente en el system prompt del chat cuando se detectan queries arquitectónicas, sin intervención del usuario.

Este documento presenta un análisis técnico exhaustivo de la arquitectura de ALEJANDRÍA: el corpus de 59 nodos distribuidos en dos fases, el pipeline de ingestión con embeddings Gemini, el motor de búsqueda semántica con algoritmo híbrido, la inyección de contexto en el chat route, las tres herramientas de extensión expuestas al LLM y la interfaz de usuario gold-themed en /alejandria.

1. Introducción y Contexto Arquitectónico

1.1 Posición de ALEJANDRÍA en el Ecosistema SOFIAA OS

SOFIAA OS es un ecosistema de inteligencia artificial modular diseñado para agencias de marketing y unidades institucionales. Sus módulos principales son: SOFIAA (núcleo conversacional y orquestador), N.E.X.O. (Knowledge Graph semántico y sistema nervioso de memoria), PROMETEO (motor de Growth Intelligence), NORA (observadora episódica del usuario), ATENA (motor de análisis científico Lean Six Sigma / DMAIC), TEC BII (gestión institucional), HERMES (capa de ejecución de acciones) y ALEJANDRÍA (sistema de autoconocimiento de ingeniería).

Dentro de esta arquitectura, ALEJANDRÍA ocupa un lugar ontológicamente único: es el único módulo cuyo dominio de conocimiento es el propio sistema del que forma parte. Mientras N.E.X.O. almacena conocimiento sobre el usuario y su entorno, ALEJANDRÍA almacena conocimiento sobre SOFIAA OS mismo —su historia, sus decisiones, sus módulos y su evolución. Es la memoria declarativa de ingeniería del sistema.

1.2 El Problema del Autoconocimiento en Sistemas de IA

Los modelos de lenguaje de gran escala presentan un déficit estructural cuando se les pregunta sobre sistemas propietarios: no tienen acceso a la documentación interna, confunden detalles de versiones, alucinan características no implementadas y son incapaces de razonar sobre el estado actual de un codebase que evoluciona constantemente. Este problema se agudiza en sistemas como SOFIAA OS, que llevan más de 300 sprints de desarrollo documentado.

ALEJANDRÍA resuelve este problema implementando un patrón de Retrieval-Augmented Generation (RAG) especializado: en lugar de depender de la memoria paramétrica del modelo, extrae el contexto relevante desde documentos estructurados —el corpus— y lo inyecta en el system prompt antes de cada respuesta. La respuesta del modelo es entonces fundamentada en fuente de verdad, no en interpolación estadística.

1.3 Historia de Sprints

ALEJANDRÍA fue construida en seis sprints principales: AJ-0 (Schema + Manifest + Firestore helpers), AJ-1 (Pipeline de ingestión con embeddings Gemini), AJ-2 (Motor de búsqueda semántica + endpoint REST), AJ-3 (Inyección de contexto en chat route), AJ-4 (Extension tools para el LLM) y AJ-5 (Interfaz de usuario /alejandria). La construcción del corpus requirió trabajo previo: reparación de JSONs rotos y un extractor determinístico que procesó los documentos fuente sin llamadas a APIs externas.

2. Filosofía de Diseño — El Problema del Autoconocimiento

2.1 Principios de Diseño

Corpus como Fuente de Verdad: toda la información que ALEJANDRÍA proporciona debe originarse en el corpus de 59 documentos JSON, no en la memoria paramétrica del LLM. Cuando el corpus y el modelo discrepan, el corpus gana. Esto se garantiza estructuralmente al inyectar el contexto del corpus en el system prompt antes de que el modelo genere cualquier respuesta.

Búsqueda Híbrida Multi-señal: ninguna señal sola es suficiente. La similitud semántica (coseno sobre embeddings) es precisa pero computacionalmente costosa y puede fallar con términos técnicos específicos. El scoring por keywords es rápido y preciso para términos exactos pero ciego al significado. El reinforcement por uso captura qué documentos han sido históricamente útiles. ALEJANDRÍA combina las tres señales con pesos calibrados.

Zero-Friction Context Injection: el usuario no necesita invocar explícitamente a ALEJANDRÍA. El sistema detecta automáticamente cuando la conversación versa sobre arquitectura, módulos o historia de SOFIAA OS, y enriquece el contexto sin interrupción. El usuario recibe respuestas fundamentadas sin cambiar su flujo de trabajo.

Reinforcement Learning por Uso: cada nodo del corpus lleva un contador reinforceCount que se incrementa cada vez que ese nodo se inyecta en un prompt. Con el tiempo, esto crea un mapa de cuáles documentos son más consultados, que se usa como señal adicional en el scoring. Los nodos más útiles tienden a ser recuperados más frecuentemente, creando un ciclo de retroalimentación positivo.

2.2 Decisiones Arquitectónicas Clave

La elección de Gemini gemini-embedding-001 como modelo de embeddings se tomó después de explorar múltiples alternativas. Groq —el proveedor principal de inferencia de SOFIAA— no ofrece endpoint de embeddings. OpenAI ofrece text-embedding-3-small con 1536 dimensiones pero requiere una clave API adicional. Gemini ofrece gemini-embedding-001 con 768 dimensiones a través de la misma API key ya configurada en el sistema, minimizando la superficie de credenciales.

La decisión de excluir el campo texto_embedding de Firestore (manteniéndolo solo en los archivos JSON locales del corpus) fue tomada tras experimentar el error 'Transaction too big' al intentar escribir vectores de 768 floats (≈6KB/doc) junto con el campo de texto de hasta 1,500 palabras por nodo. La solución: texto_embedding vive en disco local (alejandria_corpus/), el vector embedding persiste en Firestore, y la búsqueda opera exclusivamente sobre los datos ya cargados en Firestore.

El batch size de 10 nodos (en lugar del límite teórico de 500 de Firestore) fue determinado empíricamente. Cada nodo con un vector de 768 floats ocupa aproximadamente 6KB. Un batch de 10 nodos ≈ 60KB, que respeta el límite de tamaño de transacción de Firestore (1MB por batch write). Batches más grandes causaron el error 'Transaction too big' documentado durante el sprint AJ-1.

3. Arquitectura del Corpus

3.1 Estructura del Corpus

El corpus de ALEJANDRÍA está organizado en dos fases, reflejando la evolución del proyecto SOFIAA OS:

Fase	Documentos	Período	Contenido Principal
fase_1	~35 nodos	2025 – Q1 2026	Fundación del sistema: primeros sprints, decisiones de arquitectura base, especificaciones iniciales de NEXO, PROMETEO, TEC BII
fase_2	~24 nodos	Q2 – Q3 2026	Expansión: ATENA (Six Sigma), HERMES (ejecución), TEC BII v2, ALEJANDRÍA, sprints avanzados

3.2 Tipos de Nodo (AlejandriaNodeType)

El corpus contiene siete tipos de documentos, cada uno con un propósito epistemológico distinto:

Tipo	Ícono	Descripción	Casos de uso
sprint		Reporte de un sprint completado o en curso	¿Qué se hizo en el Sprint A-9? ¿Cuándo se construyó HERMES?
decision_arquitectura		Decisión técnica con alternativas y justificación	¿Por qué se eligió Groq? ¿Por qué Firestore y no PostgreSQL?
brainstorming		Lluvia de ideas, propuestas, exploraciones conceptuales	¿Cómo surgió la idea de ALEJANDRÍA? ¿Qué alternativas se exploraron?
especificacion_modulo		Especificación técnica de un módulo SOFIAA	¿Cómo funciona el Decay Engine de NEXO? ¿Cuál es el schema de ATENA?
experimento		Hipótesis, resultado, aprendizaje	¿Qué se intentó con el semantic cache? ¿Qué falló en el Rate Limiter?
hito		Milestone relevante del proyecto	¿Cuándo se hizo el primer deploy a Vercel? ¿Cuándo se alcanzó el Sprint 300?
idea		Concepto suelto aún no implementado	¿Qué ideas están pendientes para LIVE SDK? ¿Qué se planea para NORA?

3.3 Módulos del Sistema (AlejandriaModulo)

Cada nodo del corpus declara los módulos del sistema que afecta, permitiendo filtrado y búsqueda por dominio:

Módulo	Color ALEJANDRÍA	Descripción
SOFIAA	#7c3aed	Núcleo conversacional y orquestador del ecosistema
NEXO	#0891b2	Knowledge Graph semántico — sistema nervioso de memoria del usuario
PROMETEO	#f97316	Motor de Growth Intelligence — Brand DNA, Creative Lab, Director Autónomo
NORA	#ec4899	Observadora episódica del usuario — bienestar y seguimiento emocional
HERMES	#a78bfa	Capa de ejecución de acciones — conectores externos con Human-in-the-Loop
ATENA	#60a5fa	Motor científico Lean Six Sigma — DMAIC, SPC, AMEF, análisis estadístico
TEC_BII	#34d399	Gestión institucional — proyectos, briefs, equipo, ROI
ALEJANDRIA	#fbbf24	Autoconocimiento de ingeniería — el presente documento
LIVE_SDK	#94a3b8	SDK de integración en tiempo real — embebido en apps externas (futuro)

3.4 Extractor Determinístico

El corpus no fue generado con llamadas a APIs de LLM (decisión deliberada para garantizar reproducibilidad y cero dependencia de cuotas externas). En su lugar, se construyó un extractor determinístico (`alejandria-extract.mjs`) que parsea cada documento fuente y extrae los campos del schema mediante reglas heurísticas: detección de tipo por nombre de archivo, extracción de fecha por patrón regex, identificación de módulos por keywords presentes en el contenido.

Esta decisión arquitectónica tiene implicaciones importantes: el corpus es 100% reproducible sin costos de API, puede regenerarse en segundos ante cualquier cambio en la documentación fuente, y no introduce sesgos de interpretación de LLM en los metadatos del corpus.

4. Sistema de Tipos — Schema Completo

4.1 AlejandriaNode — Documento Principal

La entidad central del sistema. Almacenada en Firestore bajo `users/{uid}/alejandria_nodos/{nodeId}`. Sus campos se agrupan en seis categorías:

Identificación

```
id:          string;    // formato: {tipo}_{modulo}_{descripcion}_{fecha}
uid:         string;    // propietario (uid de Firebase Auth)
```

Clasificación

```
tipo:        AlejandriaNodeType;
fecha:       string;    // YYYY-MM-DD del documento original
fase_corpus: string;    // 'fase_1' | 'fase_2'
```

Contenido Semántico

```
titulo:      string;    // título del documento
resumen:     string;    // 2-4 oraciones densas — lo que SOFIAA lee en el
prompt
```

Clasificación Cruzada

```
modulos_afectados: AlejandriaModulo[];
sprint_referencia:  string | null;    // 'A-9', 'H-4', null
version_sofiaa:     string | null;    // '1.0', '1.1.4', null
```

Conocimiento Estructurado

```
decisiones:        AlejandriaDecision[];
conceptos_clave:    AlejandriaConcepto[];
hitos:              AlejandriaHito[];
preguntas_que_responde: string[];
tags:               string[];
```

Búsqueda Semántica

```
texto_embedding: string;    // texto comprimido, max ~1,500 palabras (local
only)
embedding?:      number[];  // vector 768-dim generado por Gemini (en
Firestore)
```

Proveniencia y Telemetría

```
documento_original: string;    // nombre del archivo fuente
procesado_por:       string;    // 'claude' | 'extractor-deterministico'
reinforceCount:      number;    // veces que este nodo fue recuperado en contexto
createdAt:           number;    // epoch ms
updatedAt:           number;    // epoch ms
```

4.2 Tipos Auxiliares

AlejandroDecision

Representa una decisión técnica documentada dentro de un nodo. Contiene: decision (qué se decidió), contexto (problema que la motivó), alternativas_descartadas (opciones descartadas con razones), justificacion (razón principal) y consecuencias (qué cambió como resultado).

AlejandroConcepto

Concepto técnico documentado en contexto SOFIAA. Contiene: concepto (nombre, ej: 'Event Sourcing', 'Knowledge Graph'), definicion (en contexto SOFIAA, no genérica) y relacion_modulos (módulos del sistema relacionados).

AlejandroHito

Milestone del proyecto. Contiene: hito (descripción de lo logrado o intentado), resultado ('exitoso' | 'fallido' | 'parcial' | 'pendiente') y aprendizaje (qué se aprendió).

4.3 AlejandroSearchResult

Resultado de búsqueda devuelto por el motor semántico. Contiene el nodo completo y su score normalizado (0–1). El score combina similitud coseno semántica, scoring por keywords y reinforcement por uso previo.

4.4 AlejandroStats

Estadísticas del corpus para la UI del Centro de Mando. Contiene: totalNodos, porTipo (mapa tipo → count), modulosCubiertos (array de módulos con al menos un nodo), decisionesTotal (suma de todas las decisiones documentadas en todos los nodos) y ultimaActualizacion (timestamp del nodo más recientemente actualizado).

5. Pipeline de Ingestión (AJ-1)

5.1 Script de Ingestión

El pipeline de ingestión (`scripts/ingest-alejandria.mjs`) es un script Node.js ESM que ejecuta cuatro fases en secuencia: carga de configuración desde `.env.local`, inicialización de Firebase Admin con credenciales de service account (base64), generación de embeddings Gemini para cada nodo, y escritura batch en Firestore.

El script es invocable de dos formas: `node scripts/ingest-alejandria.mjs {uid}` o `ALEJANDRIA_UID={uid} node scripts/ingest-alejandria.mjs`. También se expone como npm script bajo el comando `alejandria:ingest` para facilitar re-ingestiones.

5.2 Fase 1 — Lectura del Corpus

La función `getAllJsonFiles()` recorre recursivamente el directorio `alejandria_corpus/` y retorna todos los archivos `.json` que no comiencen con underscore (los que comienzan con `_` son índices o archivos de configuración). La validación `isAlejandriaSchema()` verifica que cada documento tenga los campos mínimos requeridos (`id`, `tipo`, `titulo`, `resumen`) antes de procesarlo.

5.3 Fase 2 — Generación de Embeddings Gemini

El sistema de embeddings usa el modelo `gemini-embedding-001` a través de la API v1 de Google Generative Language. La elección de la versión v1 (no v1beta) fue resultado de depuración: el modelo no existía en v1beta. El campo `texto_embedding` de cada nodo —un resumen comprimido del contenido del documento, máximo ~9,000 caracteres— se envía al endpoint:

```
POST https://generativelanguage.googleapis.com/v1/models/gemini-embedding-001:embedContent
```

El modelo retorna un vector de 768 dimensiones de punto flotante. El sistema implementa control de rate limits con tres mecanismos:

Mecanismo	Valor	Propósito
EMBED_DELAY_MS	1,500 ms	Pausa entre requests consecutivos para respetar el rate limit de la API
EMBED_MAX_RETRIES	3 intentos	Reintentos automáticos ante error 429 (Too Many Requests)
Backoff exponencial	5s / 10s / 15s	Espera creciente entre reintentos: intento 1 → 5s, intento 2 → 10s, intento 3 → 15s

5.4 Fase 3 — Escritura en Firestore

Los nodos se escriben en Firestore mediante la función `batchWrite()` con dos restricciones críticas:

Batch size = 10 (no 499): los vectores de 768 floats ocupan ~6KB por nodo. Un batch de 499 nodos superaría el límite de tamaño de transacción de Firestore (1MB). La experimentación durante AJ-1 confirmó que batches de 10 nodos (~60KB) son seguros y nunca generan el error 'Transaction too big'.

Exclusión de `texto_embedding`: el campo `texto_embedding` (hasta 1,500 palabras por nodo) se excluye explícitamente de la escritura en Firestore. Solo el vector `embedding` (768 floats) persiste en la nube. El texto permanece en los archivos JSON locales del corpus, disponibles para re-ingestión si se necesita regenerar los vectores.

5.5 Manejo de Errores y Resumen

El script implementa contadores de estadísticas (stats) que registran nodos válidos, inválidos, con embedding y con errores de embedding. Al finalizar, imprime un resumen completo con el path de Firestore donde pueden verificarse los nodos escritos.

6. Motor de Búsqueda Semántica (AJ-2)

6.1 Arquitectura del Motor

El motor de búsqueda (`src/lib/alejandria/search.ts`) implementa una búsqueda semántica híbrida en tres fases: generación del embedding del query, scoring multi-señal sobre todos los nodos del corpus, y ordenamiento y filtrado de resultados. Las tres funciones exportadas son:

```
generateQueryEmbedding(query) → Promise<number[] | null>
semanticSearchAlejandria(uid, query, limit) →
Promise<AlejandriaSearchResult[]>
getSemanticAlejandriaContext(uid, query, limit) → Promise<AlejandriaNode[]>
```

6.2 Fase 1 — Embedding del Query

La función `generateQueryEmbedding()` envía el query (truncado a 9,000 caracteres para respetar el límite de Gemini) al mismo endpoint que usa el pipeline de ingestión. El embedding del query y los embeddings del corpus comparten el mismo espacio vectorial, lo que garantiza que la similitud coseno sea significativa.

La función retorna `null` (no lanza excepción) si la API key no está configurada o si la llamada falla. Este diseño permite graceful degradation: el sistema sigue funcionando en modo keyword-only cuando el servicio de embeddings no está disponible.

6.3 Fase 2 — Similitud Coseno

La función `cosine()` implementa el producto punto normalizado entre dos vectores:

```
cosine(a, b) = dot(a,b) / (|a| × |b|)
```

La implementación en TypeScript itera sobre las 768 dimensiones del vector acumulando el producto punto y las normas. El denominador se verifica para evitar división por cero (un vector de norma cero retorna similitud 0). El rango de salida es `[0, 1]` para embeddings de texto normalizados.

6.4 Fase 3 — Scoring Híbrido

El score final de cada nodo combina tres señales con pesos calibrados:

Señal	Peso	Función	Rango
Similitud semántica (coseno)	0.70 (70%)	<code>cosine(queryEmbedding, node.embedding)</code>	0.0 – 1.0
Keyword boost	0.20 (20%)	<code>keywordScore(node, query)</code>	0.0 – 1.0
Reinforcement por uso	0.10 (10%)	<code>node.reinforceCount / maxReinforce</code>	0.0 – 1.0

La fórmula completa con embeddings disponibles:

```
score = semantic × 0.70 + keyword × 0.20 + reinforced × 0.10
```

Cuando el embedding del query no está disponible (API no configurada o fallo), el scoring degrada automáticamente a modo keyword + reinforcement:

```
score = keyword × 0.80 + reinforced × 0.20
```

6.5 Keyword Boost

La función `keywordScore()` tokeniza el query (split por espacios, filtra tokens de menos de 3 caracteres) y busca cada token en el texto concatenado de titulo, resumen, tags, `modulos_afectados` y `preguntas_que_responde` del nodo. El score es la fracción de tokens del query que aparecen en el nodo:

```
keyword_score = tokens_encontrados / tokens_totales_query
```

6.6 Filtrado de Resultados

Los resultados se ordenan descendientemente por score y se devuelven los top-K (por defecto `MAX_RESULTS=8`). Un filtro adicional excluye nodos con score cero —aquellos sin ninguna relación semántica ni léxica con el query— para mantener la relevancia del contexto inyectado.

7. Inyección de Contexto en el Chat (AJ-3)

7.1 Detección Automática de Queries Arquitectónicas

El chat route (`src/app/api/chat/route.ts`) implementa un detector de queries relacionadas con la arquitectura de SOFIAA OS mediante la función `isAlejandriaQuery()`. El detector normaliza el texto del último mensaje del usuario (`toLowerCase`) y verifica si contiene alguno de los triggers configurados:

Categoría	Triggers
Módulos del sistema	nexo, prometeo, hermes, atena, tec bii, tec-bii, nora, live sdk, alejandría, alejandria
Preguntas de funcionamiento	cómo funciona, como funciona, qué es, que hace, cómo se implementó
Preguntas de arquitectura	arquitectura, diseño, estructura, infraestructura
Preguntas históricas	sprint, decisión, decidió, por qué se eligió, cuándo se construyó
Identidad del sistema	sofiaa os, historia del sistema, módulos

7.2 Flujo de Inyección

Cuando `isAlejandriaQuery()` retorna `true`, el chat route invoca `getAlejandriaContext(uid, query)` que a su vez llama a `getSemanticAlejandriaContext()` para obtener los nodos más relevantes. Los node IDs recuperados se almacenan en `alejandriaNodeIds` para el refuerzo post-stream.

El contexto se materializa como un bloque de texto formateado mediante `buildAlejandriaBlock()` que se inyecta en el system prompt entre el bloque de NEXO (memoria del usuario) y el bloque cognitivo:

```
MEMORIA ALEJANDRÍA — Tu historia de ingeniería
Documentos recuperados del Knowledge Graph de SOFIAA OS:

• [SPRINT] Sprint A-9 — Wizard de Proyectos ATENA
Módulos: ATENA · Sprint: A-9
[resumen del nodo]
```

7.3 Refuerzo Post-Stream (`reinforceAlejandriaNodes`)

Tras la finalización del stream de respuesta —tanto en modo agente como en modo conversacional normal— el chat route llama a `reinforceAlejandriaNodes()` de forma asíncrona (usando `.catch(() => {})` para no bloquear la respuesta). Esta función incrementa el campo `reinforceCount` de cada nodo recuperado en el prompt usando `FieldValue.increment(1)` de Firestore, garantizando operaciones atómicas sin condiciones de carrera.

El refuerzo opera en batches de máximo `REINFORCE_BATCH=20` nodos. En la práctica, el número de nodos inyectados por prompt es de 4 a 8, por lo que el batch siempre cabe en una sola operación de escritura.

8. Extension Tools (AJ-4)

8.1 Arquitectura de Herramientas

ALEJANDRÍA expone tres herramientas de extensión al LLM a través de la interfaz `ExtensionToolRegistry` definida en `@/types/sofiaa-platform`. Este registry sigue el patrón establecido por ATENA y TEC BII: un array `tools` con las definiciones JSON Schema de cada herramienta, y un handler asíncrono que despacha la ejecución según el nombre de la herramienta invocada.

8.2 buscar_en_alejandria

La herramienta de búsqueda principal. Acepta un query de texto libre en español y un parámetro `limit` opcional (1-10, default 5). Internamente llama a `semanticSearchAlejandria()` y formatea los resultados como texto legible para el LLM:

Campo	Tipo	Descripción
query	string (required)	Pregunta o tema a buscar. Texto libre en español.
limit	number (optional)	Máximo de resultados (1–10). Default: 5.

El output formateado incluye para cada resultado: número de orden, tipo en mayúsculas, título, porcentaje de relevancia (`score × 100`), fecha, sprint de referencia, módulos afectados y resumen completo. Este formato está optimizado para que el LLM pueda citar la fuente con precisión.

8.3 listar_modulos_sofiaa

Herramienta de catálogo estático. No acepta parámetros. Retorna el catálogo completo de los 9 módulos del sistema con sus descripciones. El catálogo está definido inline en `tools.ts` como un `Record<AlejandriaModulo, string>`, lo que garantiza que esté siempre sincronizado con el tipo `AlejandriaModulo` del schema.

Esta herramienta es útil cuando el usuario pregunta '¿cuántos módulos tiene SOFIAA?' o '¿qué hace HERMES?' sin necesidad de consultar el corpus —el catálogo es determinístico y no requiere búsqueda vectorial.

8.4 buscar_decision_arquitectura

Filtro especializado en decisiones de arquitectura. Más específico que `buscar_en_alejandria`: solo retorna nodos de tipo `decision_arquitectura`. Acepta dos parámetros opcionales:

Parámetro	Tipo	Descripción
query	string (optional)	Búsqueda semántica libre sobre el tipo de decisión.
modulo	string (optional)	Filtrar por módulo: SOFIAA, NEXO, PROMETEO, NORA, HERMES, ATENA, TEC_BII, ALEJANDRIA, LIVE_SDK.

La lógica interna combina dos estrategias: si se provee un módulo, se usa `getNodesByModulo()` para filtrar por módulo y luego se filtra por tipo `decision_arquitectura`. Si se provee un query, los resultados se re-ranquean semánticamente via `semanticSearchAlejandria()`. El output incluye para cada decisión las primeras dos entradas del campo `decisiones[]` con su justificación truncada a 80 caracteres.

8.5 Registro en la Extensión

Las herramientas se registran en `src/extensions/alejandria/index.ts` importando `alejandriaTools` desde `tools.ts` y asignándolo al campo opcional `tools` de la `SofiaaExtension`. El `ExtensionRegistry` del core detecta automáticamente la presencia del registry y lo expone al LLM en el system prompt de la sesión correspondiente.

9. Interfaz de Usuario (AJ-5)

9.1 Paleta y Tema Visual

La interfaz de ALEJANDRÍA usa una paleta amber/gold (#fbbf24) que la distingue visualmente de los demás módulos del sistema: NEXO usa cyan (#0891b2), PROMETEO usa naranja (#f97316), ATENA usa violeta (#60a5fa), y HERMES usa lavanda (#a78bfa). El fondo base es near-black con tinte cálido (#0a0800 / #0d0b00), creando una atmósfera de 'archivo antiguo iluminado' que evoca la biblioteca de Alejandría original.

9.2 Centro de Mando (/alejandria)

El Centro de Mando es la página principal del módulo. Muestra cuatro KPI cards con las estadísticas del corpus (total nodos, módulos cubiertos, decisiones documentadas, fecha de última ingestión), un panel de distribución por tipo que muestra visualmente cuántos nodos de cada tipo existen en el corpus, un panel de módulos documentados con chips gold para cada módulo presente en el corpus, y el buscador semántico inline.

El buscador implementa debounce de 500ms: la búsqueda se lanza 500ms después de que el usuario deja de escribir, evitando requests en cada keystroke. La UI muestra un indicador 'buscando...' durante la llamada API, y los resultados se presentan como NodeCards con tipo, fecha, módulos, sprint de referencia y un badge  con el reinforceCount si el nodo ha sido consultado previamente.

9.3 Explorador de Nodos (/alejandria/nodos)

El explorador permite navegar el corpus con filtros por tipo de nodo y por módulo. Los filtros se implementan como select elements que operan sobre los resultados ya cargados en memoria (client-side filtering), sin llamadas API adicionales. La carga inicial usa la búsqueda semántica con un query amplio para obtener una muestra representativa del corpus ordenada por relevancia.

9.4 Layout Responsive

El layout (src/app/alejandria/layout.tsx) implementa el mismo patrón responsive que ATENA: en desktop (≥768px) se renderiza un sidebar fijo de 224px con el nav de tres items. En móvil (<768px) se muestra una top bar sticky con el nombre del módulo y un botón de hamburger que abre un sidebar deslizable con overlay oscuro semitransparente. El PageGuard de TEC BII controla el acceso al módulo.

9.5 Integración con la Navegación Principal

ALEJANDRÍA está integrada en el nav rápido de la pantalla principal de SOFIAA (page.tsx) mediante un chip gold junto a los chips de TEC Bii, PROMETEO y ATENA. El chip usa los colores ambient de la paleta: rgba(251, 191, 36, 0.13) de fondo y rgba(251, 191, 36, 0.30) de borde.

10. Esquema Firestore

10.1 Colección Principal

```
users/  
  {uid}/  
    alejandria_nodos/           ← Corpus vectorizado  
      {nodeId}/                 ← Documento AlejandriaNode  
        id, uid, tipo, fecha  
        titulo, resumen  
        modulos_afectados[], sprint_referencia, version_sofiaa  
        decisiones[], conceptos_clave[], hitos[]  
        preguntas_que_responde[], tags[]  
        embedding[]             ← Vector 768-dim (Gemini)  
        documento_original, procesado_por  
        reinforceCount, createdAt, updatedAt
```

Nótese que el campo `texto_embedding` está explícitamente ausente de Firestore. Solo vive en los archivos JSON locales del corpus (`alejandria_corpus/`). Esta separación es una decisión de diseño arquitectónico: el texto de búsqueda (largo, hasta 1,500 palabras) no necesita persistir en la nube porque se usa únicamente durante la ingestión para generar el vector.

10.2 Índices Recomendados

Para las queries de la UI y del motor de búsqueda, se recomiendan los siguientes índices compuestos en Firestore:

Colección	Campos	Query que soporta
alejandria_nodos	uid ASC + tipo ASC + fecha DESC	Filtrado por tipo ordenado cronológicamente
alejandria_nodos	uid ASC + reinforceCount DESC	Top nodos más consultados
alejandria_nodos	uid ASC + modulos_afectados ARRAY_CONTAINS + fecha DESC	Nodos por módulo
alejandria_nodos	uid ASC + sprint_referencia ASC	Nodos de un sprint específico
alejandria_nodos	uid ASC + updatedAt DESC	Último nodo actualizado (stats)

10.3 Reglas de Seguridad Firestore

Los nodos de ALEJANDRÍA son documentos privados del usuario. Las reglas de Firestore restringen el acceso estrictamente al propietario:

```
match /users/{uid}/alejandria_nodos/{nodeId} {  
  allow read: if request.auth != null && request.auth.uid == uid;  
  allow write: if false; // solo Firebase Admin SDK  
}
```

Las escrituras son exclusivamente via Firebase Admin SDK (service account con credenciales en `FIREBASE_SERVICE_ACCOUNT_BASE64`). El cliente React nunca escribe directamente en

alejandria_nodos. Esta separación garantiza la integridad del corpus: solo el pipeline de ingestión puede modificar los nodos.

11. Seguridad y Control de Acceso

11.1 Autenticación en los Endpoints

Los dos endpoints REST de ALEJANDRÍA (POST /api/alejandria/search y GET /api/alejandria/stats) implementan autenticación Firebase mediante el mismo patrón que HERMES: extracción del Bearer token del header Authorization, verificación con `getAuth(getAdminApp()).verifyIdToken(token)`, y extracción del uid verificado. Cualquier request sin token válido retorna 401 sin ejecutar lógica de negocio.

11.2 Aislamiento por UID

Todo acceso al corpus en Firestore es parametrizado por uid. La función `alejandriaNodesCol(uid)` construye el path `users/{uid}/alejandria_nodos`, garantizando que un usuario autenticado solo pueda acceder a su propio corpus. No existe ninguna ruta de acceso cross-user en la arquitectura.

11.3 Credenciales de Firebase Admin

El pipeline de ingestión (`scripts/ingest-alejandria.mjs`) usa Firebase Admin SDK con credenciales de service account. La clave se configura como base64 en la variable de entorno `FIREBASE_SERVICE_ACCOUNT_BASE64`. Las claves API nunca se comparten en el chat —deben ingresarse via nano en terminal o mediante la interfaz de Vercel. El archivo `.env.local` está en `.gitignore` y nunca se incluye en commits.

11.4 Gemini API Key

La clave `GEMINI_API_KEY` se usa únicamente en dos contextos: el pipeline de ingestión (AJ-1) para generar embeddings del corpus, y el motor de búsqueda (AJ-2) para generar el embedding del query en tiempo real. En ambos casos la clave se lee exclusivamente desde `process.env`. El motor de búsqueda implementa graceful degradation: si la clave no está disponible, opera en modo keyword-only sin lanzar excepciones.

12. Capa de API REST

12.1 POST /api/alejandria/search

Endpoint principal de búsqueda semántica. Server-side only (Next.js Route Handler).

Campo	Valor
Método	POST
Autenticación	Bearer {Firebase ID Token}
Body	{ query: string; limit?: number }
Validación	query mínimo 2 caracteres; limit máximo 20
Respuesta exitosa	{ success, query, results: [...], total, took }
Error 401	{ success: false, error: 'No autorizado' }
Error 400	{ success: false, error: 'query requerido (mínimo 2 caracteres)' }

Cada elemento de results contiene: id, tipo, titulo, resumen, fecha, modulos_afectados, tags, sprint_referencia, reinforceCount y score (redondeado a 3 decimales).

12.2 GET /api/alejandria/stats

Endpoint de estadísticas del corpus. Retorna el objeto AlejandriaStats completo para la UI del Centro de Mando.

Campo	Valor
Método	GET
Autenticación	Bearer {Firebase ID Token}
Respuesta exitosa	{ success: true, stats: AlejandriaStats }
Error 401	{ success: false, error: 'No autorizado' }

AlejandriaStats incluye: totalNodos (int), porTipo (Record<AlejandriaNodeType, number>), modulosCubiertos (AlejandriaModulo[]), decisionesTotal (int suma de todas las decisiones en todos los nodos), ultimaActualizacion (epoch ms del nodo más reciente).

13. Algoritmo de Scoring Híbrido

13.1 Diseño del Algoritmo

El algoritmo de scoring híbrido de ALEJANDRÍA fue diseñado para resolver las limitaciones individuales de cada señal disponible:

Señal	Fortaleza	Debilidad
Similitud coseno	Captura significado semántico; entiende sinónimos y paráfrasis	Falla con términos técnicos muy específicos (acrónimos, nombres propios); costosa computacionalmente
Keyword boost	Precisa para términos exactos; cero latencia (no requiere API)	Ciega al significado; 'NEXO' y 'Knowledge Graph' no se relacionan
Reinforcement	Captura utilidad histórica real; se adapta al comportamiento del usuario	Sesgada hacia nodos antiguos; nueva ingestión empieza en cero

La combinación con pesos 0.70 / 0.20 / 0.10 refleja la jerarquía de señales: la semántica domina (el 70% del score viene del embedding), pero los keywords garantizan que términos técnicos exactos no se pierdan, y el reinforcement actúa como un suave prior bayesiano sobre la utilidad histórica.

13.2 Calibración de Pesos

Los pesos fueron calibrados con base en la naturaleza del corpus. Los documentos de ALEJANDRÍA tienen vocabulario técnico muy específico (nombres de módulos, patrones de diseño, tecnologías concretas) donde el keyword matching es muy efectivo. El peso de 0.20 para keywords (vs. 0.65 en NEXO que solo usa semántica) refleja esta diferencia.

El peso de 0.10 para reinforcement es deliberadamente conservador: se quiere que los nodos nuevos (reinforceCount=0) compitan en igualdad de condiciones con nodos frecuentemente consultados, evitando que el corpus se 'calcifique' alrededor de los documentos más antiguos.

13.3 Comportamiento en Casos Límite

Sin embedding disponible: el sistema degrada a $\text{keyword} \times 0.80 + \text{reinforced} \times 0.20$. Esta degradación es explícita en el código (comentario documentado) y garantiza que el sistema siempre retorne resultados, incluso si el servicio de Gemini no está disponible.

Score cero: nodos con score exactamente 0 se filtran del resultado final. Un score de 0 indica que el nodo no tiene ninguna relación semántica ni léxica con el query —incluirlo generaría ruido en el contexto inyectado al LLM.

Normalización de reinforceCount: el campo se normaliza dividiendo por el máximo del corpus: $\text{reinforced} = \text{reinforceCount} / \max(\text{reinforceCounts})$. Si todos los nodos tienen reinforceCount=0, el denominador se establece en 1 para evitar división por cero.

14. Historia de Sprints

Sprint	Nombre	Entregables Principales
Pre-AJ	Corpus Prep	Reparación de JSONs rotos; extractor determinístico alejandria-extract.mjs; generación de 59 nodos JSON desde documentación fuente
AJ-0	Schema y Fundación	AlejandriaNode interface completa; AlejandriaExtension manifest; helpers Firestore (CRUD, getNodesByTipo, getNodesByModulo, getNodesBySprint, reinforceAlejandriaNodes, getAlejandriaStats); alejandriaNodesCol()
AJ-1	Pipeline de Ingestión	scripts/ingest-alejandria.mjs con embeddings Gemini gemini-embedding-001; rate limiting (1,500ms delay, 3 reintentos, backoff exponencial); batch write de 10 nodos; npm script alejandria:ingest; 59/59 nodos ingestados
AJ-2	Motor de Búsqueda	src/lib/alejandria/search.ts con cosine similarity + keyword boost + reinforcement; generateQueryEmbedding(); semanticSearchAlejandria(); getSemanticAlejandriaContext(); POST /api/alejandria/search autenticado
AJ-3	Inyección en Chat	isAlejandriaQuery() con triggers de keywords arquitectónicas; buildAlejandriaBlock(); inyección en system prompt; reinforceAlejandriaNodes() post-stream asíncrono
AJ-4	Extension Tools	ExtensionToolRegistry con 3 herramientas: buscar_en_alejandria, listar_modulos_sofiaa, buscar_decision_arquitectura; handler de despacho; registro en alejandriaExtension
AJ-5	UI /alejandria	GET /api/alejandria/stats; layout gold con sidebar responsive; Centro de Mando (KPIs + distribución + buscador semántico + debounce); Explorador de nodos con filtros; link en nav de page.tsx

15. Roadmap

15.1 Actualización Continua del Corpus

El corpus actual (59 nodos) cubre los sprints hasta AJ-5. Cada nuevo sprint significativo de SOFIAA OS debería generar un nuevo documento JSON en `alejandria_corpus/` y re-ingestar. El pipeline de ingestión (`alejandria:ingest`) está diseñado para ser idempotente: usa `merge:true` en Firestore, por lo que re-ingestar un nodo existente actualiza sus campos sin eliminar el `reinforceCount` acumulado.

La frecuencia recomendada de actualización del corpus es al cierre de cada sprint mayor o grupo de sprints relacionados. Para sprints de alta actividad como los de este documento, se recomienda una actualización semanal.

15.2 Búsqueda por Ventana Temporal

Una capacidad natural para AJ-6 sería filtrar los resultados por rango de fechas. La búsqueda actual ignora la dimensión temporal —un sprint de julio 2026 compite en igualdad de condiciones con uno de enero 2025. Una búsqueda '¿qué decidimos esta semana sobre embeddings?' debería priorizar nodos recientes. La implementación requeriría agregar un parámetro `fechaDesde/fechaHasta` a `semanticSearchAlejandria()` y filtrar antes del scoring.

15.3 Administración del Corpus desde la UI

Actualmente el corpus solo puede actualizarse via el script de línea de comandos. Una interfaz de administración permitiría añadir nuevos nodos directamente desde la UI de ALEJANDRÍA, con un formulario de edición que siga el schema `AlejandriaNode` y un botón de 'Generar embedding' que invoque un nuevo endpoint `POST /api/alejandria/ingest`.

15.4 Visualización del Knowledge Graph

Los 59 nodos con sus relaciones (`modulos_afectados`, `sprint_referencia`, `tags`) forman un grafo de conocimiento. Una visualización interactiva usando `D3.js` o `Cytoscape.js` permitiría explorar visualmente las conexiones entre sprints, módulos y decisiones —similar al 'Mi Grafo' de N.E.X.O. pero sobre la historia de ingeniería del sistema.

15.5 ALEJANDRÍA como Contexto Global

Actualmente el contexto de ALEJANDRÍA se inyecta solo cuando `isAlejandriaQuery()` detecta una query arquitectónica. Una mejora de AJ-7 podría ser inyectar un contexto mínimo de ALEJANDRÍA (los 2-3 nodos más relevantes) en cada conversación, independientemente del topic. Esto daría a SOFIAA una conciencia de fondo de su propia arquitectura en todas las interacciones.

16. Conclusiones

16.1 Madurez del Sistema

ALEJANDRÍA v1.0 constituye una implementación production-ready del sistema de autoconocimiento de ingeniería para el ecosistema SOFIAA OS. Los 59 nodos del corpus están vectorizados con embeddings Gemini de 768 dimensiones y disponibles para búsqueda semántica en tiempo real. El sistema de inyección automática de contexto está activo en el chat route, y las tres herramientas de extensión están expuestas al LLM para consultas explícitas.

16.2 Fortalezas Arquitectónicas

Las decisiones de diseño más destacadas de ALEJANDRÍA son:

Corpus determinístico: el extractor determinístico garantiza que el corpus sea 100% reproducible sin dependencias de APIs de LLM. Esto elimina el riesgo de inconsistencias entre ejecuciones y reduce el costo de mantenimiento del corpus.

Graceful degradation en embeddings: el sistema funciona en modo keyword-only cuando el servicio de Gemini no está disponible. No hay punto de fallo único que deje al sistema sin capacidad de búsqueda.

Reinforcement learning por uso: el contador reinforceCount crea un mecanismo de mejora continua pasivo. Sin intervención del desarrollador, el corpus se autoorganiza para priorizar los documentos más consultados en contexto.

Separación texto/vector: mantener texto_embedding en disco local y embedding en Firestore es una decisión de diseño que resuelve el límite de tamaño de transacción de Firestore de forma elegante, sin sacrificar la capacidad de búsqueda semántica.

16.3 Métricas de Implementación

Métrica	Valor
Nodos en el corpus	59
Dimensiones de los embeddings	768 (gemini-embedding-001)
Tipos de nodo	7 (sprint, decision_arquitectura, brainstorming, especificacion_modulo, experimento, hito, idea)
Módulos documentados	9 (SOFIAA, NEXO, PROMETEO, NORA, HERMES, ATENA, TEC_BII, ALEJANDRIA, LIVE_SDK)
Peso semántica en scoring	0.70 (70%)
Peso keyword en scoring	0.20 (20%)
Peso reinforcement en scoring	0.10 (10%)
MAX_CONTEXT_NODES	8 nodos por prompt
Batch size de ingestión	10 nodos (límite de seguridad para vectores de 768 floats)
Delay entre embeddings	1,500 ms (rate limit de Gemini)
MAX_REINTENTOS de embedding	3 (backoff exponencial 5s/10s/15s)
Extension tools expuestos al LLM	3 (buscar_en_alejandria, listar_modulos_sofiaa, buscar_decision_arquitectura)
Endpoints REST	2 (POST /search, GET /stats)
Páginas de UI	3 (/alejandria, /alejandria/buscar, /alejandria/nodos)
Sprints de desarrollo	AJ-0 → AJ-5 (6 sprints + corpus prep)
Archivos TypeScript del módulo	~10 (schema, firestore, search, tools, index, pages, layout, routes)

16.4 Visión a Largo Plazo

ALEJANDRÍA está diseñada para crecer junto con SOFIAA OS. Cada sprint futuro que genere un nuevo documento JSON en el corpus extiende el conocimiento del sistema sobre sí mismo. A medida que el corpus crezca de 59 a 200+ nodos, el reinforcement learning acumulado tenderá a crear una jerarquía natural de importancia: las decisiones arquitectónicas fundamentales (elección de Firebase, de Next.js App Router, del patrón de extensiones) serán consultadas con más frecuencia y alcanzarán los primeros lugares del ranking.

En el largo plazo, ALEJANDRÍA aspira a convertirse en la base para responder la pregunta más profunda que puede hacerse un sistema de IA: '¿Quién soy?'. No en un sentido filosófico abstracto, sino en el sentido concreto y técnico que importa para un sistema de software en producción: cuáles fueron mis decisiones de diseño, qué experimentos fallé, qué compromisos de arquitectura definieron mi forma actual, y hacia dónde me dirijo.